

Trabalho Prático 1 — Etapa 1

Modelagem do Banco de Dados

Sistema de Aluguel de Veículos

Aluno: Diogo

Código de Pessoa: 1285293

C# · ASP.NET Core · Entity Framework Core · SQL Server Express

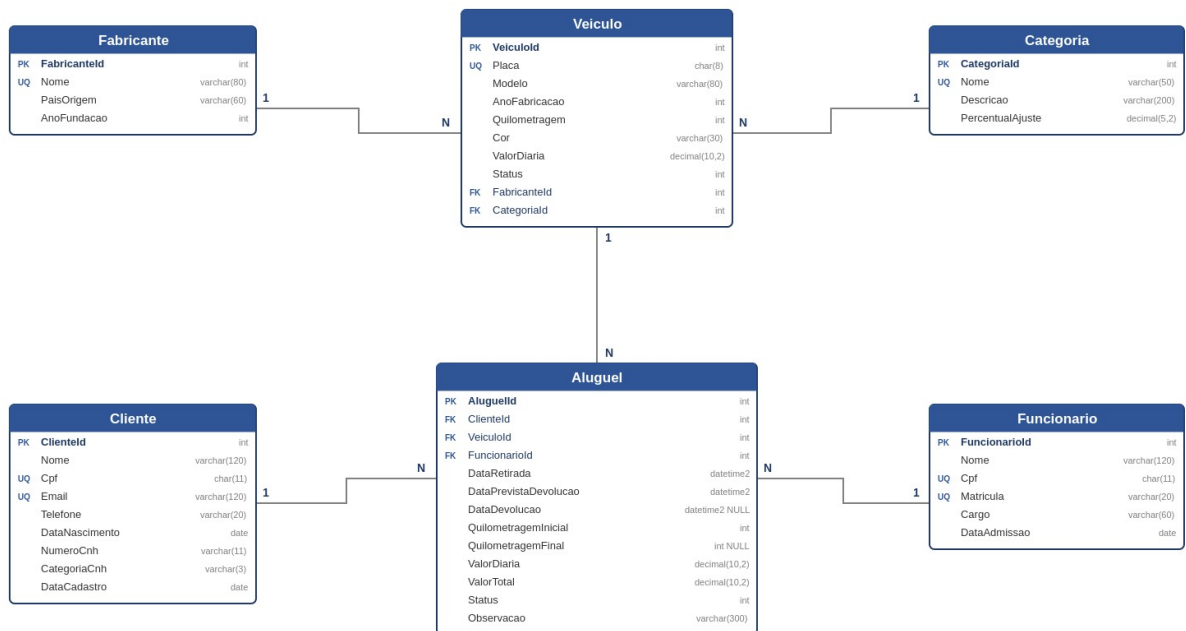
1. Objetivo da etapa

Esta etapa entrega o modelo conceitual do banco de dados da locadora de veículos, sua tradução para um esquema relacional por meio do Entity Framework Core, a implementação das classes de entidade em C# e a configuração da classe ApplicationContext responsável pelo mapeamento objeto-relacional no SQL Server.

O modelo contempla 6 entidades — uma a mais que o mínimo de 5 exigido no enunciado —, com chaves primárias, chaves estrangeiras, restrições de unicidade e restrições de verificação definidas para manter a integridade dos dados.

2. Modelo conceitual

O diagrama abaixo apresenta as entidades, seus atributos e os relacionamentos do banco LocadoraVeiculos.



PK = chave primaria | FK = chave estrangeira | UQ = restricao de unicidade

Banco: LocadoraVeiculos (SQL Server)

Figura 1 — Diagrama entidade-relacionamento do banco LocadoraVeiculos

Relacionamentos:

- Fabricante (1) → Veiculo (N): todo veículo pertence a um único fabricante.
- Categoria (1) → Veiculo (N): todo veículo pertence a uma única categoria de locação.
- Cliente (1) → Aluguel (N): um cliente pode realizar vários alugueis ao longo do tempo.
- Veiculo (1) → Aluguel (N): um veículo pode ser alugado várias vezes, em períodos distintos.
- Funcionario (1) → Aluguel (N): um funcionário registra vários contratos de aluguel.

3. Dicionário de dados

As tabelas a seguir detalham os atributos de cada entidade implementada na camada Model do projeto.

3.1. Fabricante

Marca do veículo. Um fabricante possui vários veículos na frota.

Coluna	Tipo	Restrições	Descrição
FabricanteId	int	PK, identity	Identificador do fabricante
Nome	varchar(80)	NOT NULL, UNIQUE	Nome da marca
PaisOrigem	varchar(60)	NULL	País de origem da marca
AnoFundacao	int	NULL	Ano de fundação da empresa

3.2. Categoria

Classificação de locação do veículo. Define o ajuste aplicado sobre a diária.

Coluna	Tipo	Restrições	Descrição
CategoriaId	int	PK, identity	Identificador da categoria
Nome	varchar(50)	NOT NULL, UNIQUE	Econômico, SUV, Executivo...
Descricao	varchar(200)	NULL	Detalhamento da categoria
PercentualAjuste	decimal(5,2)	DEFAULT 0	Percentual sobre o valor da diária

3.3. Veiculo

Veículo da frota. Pertence a um fabricante e a uma categoria.

Coluna	Tipo	Restrições	Descrição
VeiculoId	int	PK, identity	Identificador do veículo
Placa	char(8)	NOT NULL, UNIQUE	Placa do veículo
Modelo	varchar(80)	NOT NULL	Modelo do veículo
AnoFabricacao	int	NOT NULL, CHECK >= 1950	Ano de fabricação
Quilometragem	int	NOT NULL, CHECK >= 0	Quilometragem atual
Cor	varchar(30)	NULL	Cor predominante
ValorDiaria	decimal(10,2)	CHECK > 0	Valor base da diária
Status	int	DEFAULT 1	1-Disponível 2-Alugado 3-Manutenção 4-Inativo
FabricanteId	int	NOT NULL, FK	Referencia Fabricante
CategoriaId	int	NOT NULL, FK	Referencia Categoria

3.4. Cliente

Cliente da locadora. Possui no mínimo nome, CPF e e-mail.

Coluna	Tipo	Restrições	Descrição
ClienteId	int	PK, identity	Identificador do cliente
Nome	varchar(120)	NOT NULL	Nome completo
Cpf	char(11)	NOT NULL, UNIQUE	CPF sem formatação
Email	varchar(120)	NOT NULL, UNIQUE	E-mail de contato

Coluna	Tipo	Restrições	Descrição
Telefone	varchar(20)	NULL	Telefone de contato
DataNascimento	date	NULL	Data de nascimento
NumeroCnh	varchar(11)	NULL	Número da habilitação
CategoriaCnh	varchar(3)	NULL	Categoria da habilitação
DataCadastro	date	NOT NULL	Data de entrada na base

3.5. Funcionario

Funcionário responsável por registrar o contrato de aluguel.

Coluna	Tipo	Restrições	Descrição
FuncionarioId	int	PK, identity	Identificador do funcionário
Nome	varchar(120)	NOT NULL	Nome completo
Cpf	char(11)	NOT NULL, UNIQUE	CPF sem formatação
Matricula	varchar(20)	NOT NULL, UNIQUE	Matrícula interna
Cargo	varchar(60)	NULL	Cargo exercido
DataAdmissao	date	NOT NULL	Data de admissão

3.6. Aluguel

Contrato de locação. Vincula cliente, veículo e funcionário em um período de tempo.

Coluna	Tipo	Restrições	Descrição
AluguelId	int	PK, identity	Identificador do contrato
ClienteId	int	NOT NULL, FK	Referencia Cliente
VeiculoId	int	NOT NULL, FK	Referencia Veiculo
FuncionarioId	int	NOT NULL, FK	Referencia Funcionario
DataRetirada	datetime2	NOT NULL	Início da locação
DataPrevistaDevolucao	datetime2	NOT NULL, CHECK	Devolução combinada
DataDevolucao	datetime2	NULL	Devolução efetiva
QuilometragemInicial	int	NOT NULL	Km na retirada
QuilometragemFinal	int	NULL, CHECK	Km na devolução
ValorDiaria	decimal(10,2)	CHECK > 0	Valor da diária contratada
ValorTotal	decimal(10,2)	NOT NULL	Valor total da locação
Status	int	DEFAULT 1	1-Em andamento 2-Finalizado 3-Cancelado
Observacao	varchar(300)	NULL	Observações do contrato

4. Chaves primárias

Todas as entidades utilizam chave primária simples, do tipo inteiro, com geração automática pelo banco (identity). A definição é feita por anotação [Key] na classe e confirmada por Fluent API no ApplicationDbContext.

Tabela	Chave primária	Geração
Fabricante	FabricanteId	Identity
Categoria	CategoriaId	Identity
Veiculo	VeiculoId	Identity
Cliente	ClienteId	Identity
Funcionario	FuncionarioId	Identity
Aluguel	AluguelId	Identity

Exemplo da definição na classe de entidade e no contexto:

```
// Models/Veiculo.cs
[Key]
public int VeiculoId { get; set; }

// Data/ApplicationDbContext.cs
entity.HasKey(v => v.VeiculoId);
entity.Property(v => v.VeiculoId).ValueGeneratedOnAdd();
```

5. Chaves estrangeiras

O modelo possui cinco chaves estrangeiras, todas obrigatórias e nomeadas explicitamente no ApplicationDbContext.

Constraint	Origem	Coluna	Destino
FK_Veiculo_Fabricante	Veiculo	FabricanteId	Fabricante
FK_Veiculo_Categoria	Veiculo	CategoriaId	Categoria
FK_Aluguel_Cliente	Aluguel	ClienteId	Cliente
FK_Aluguel_Veiculo	Aluguel	VeiculoId	Veiculo
FK_Aluguel_Funcionario	Aluguel	FuncionarioId	Funcionario

Todas as chaves estrangeiras utilizam DeleteBehavior.Restrict, o que impede a exclusão de um registro que ainda possua dependentes — não é possível, por exemplo, excluir um cliente que tenha aluguéis registrados. A opção também evita os múltiplos caminhos de exclusão em cascata que o SQL Server não permite na tabela Aluguel, que referencia três tabelas diferentes.

```
// Chave estrangeira: Aluguel (N) -> Cliente (1)
entity.HasOne(a => a.Cliente)
    .WithMany(c => c.Aluguéis)
    .HasForeignKey(a => a.ClienteId)
    .HasConstraintName("FK_Aluguel_Cliente")
    .OnDelete(DeleteBehavior.Restrict);
```

6. Demais restrições de integridade

6.1. Restrições de unicidade

Índice único	Tabela	Coluna
UQ_Fabricante_Nome	Fabricante	Nome
UQ_Categoria_Nome	Categoria	Nome
UQ_Veiculo_Placa	Veiculo	Placa
UQ_Cliente_Cpf	Cliente	Cpf

Índice único	Tabela	Coluna
UQ_Cliente_Email	Cliente	Email
UQ_Funcionario_Cpf	Funcionario	Cpf
UQ_Funcionario_Matricula	Funcionario	Matricula

6.2. Restrições de verificação

Constraint	Regra
CK_Veiculo_AnoFabricacao	AnoFabricacao >= 1950
CK_Veiculo_Quilometragem	Quilometragem >= 0
CK_Veiculo_ValorDiaria	ValorDiaria > 0
CK_Aluguel_Periodo	DataPrevistaDevolucao >= DataRetirada
CK_Aluguel_Quilometragem	QuilometragemFinal IS NULL OR QuilometragemFinal >= QuilometragemInicial
CK_Aluguel_ValorDiaria	ValorDiaria > 0

6.3. Índices de apoio

A tabela Aluguel recebeu índices não únicos em ClienteId, VeiculoId e DataRetirada, colunas que serão usadas como filtro nas consultas com joins previstas para a Etapa 2.

7. Configuração da classe ApplicationContext

A classe `ApplicationContext`, em `Data/ApplicationContext.cs`, herda de `DbContext` e concentra todo o mapeamento objeto-relacional do projeto. Suas responsabilidades são:

- Expor um `DbSet` por entidade, que corresponde a uma tabela no banco;
- Configurar a conexão com o SQL Server, tanto por injeção de dependência quanto pelo método `OnConfiguring`, usado pelas ferramentas de migração;
- Definir, em `OnModelCreating`, os nomes de tabela, as chaves primárias, as chaves estrangeiras, os tipos de coluna, os tamanhos, os índices únicos e as restrições de verificação;
- Realizar a carga inicial de dados fixos por meio do método `HasData`.

7.1. DbSets e conexão

```
public class ApplicationContext : DbContext
{
    public ApplicationContext(DbContextOptions<ApplicationContext> options)
        : base(options) { }

    public DbSet<Fabricante> Fabricantes { get; set; }
    public DbSet<Categoria> Categorias { get; set; }
    public DbSet<Veiculo> Veiculos { get; set; }
    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Funcionario> Funcionarios { get; set; }
    public DbSet<Aluguel> Alugueis { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            optionsBuilder.UseSqlServer(
                "Server=localhost,1433;Database=LocadoraVeiculos;User Id=sa;" +
                "Password=SqlServer@2026;TrustServerCertificate=True;");
        }
    }
}
```

7.2. Mapeamento de uma entidade

Trecho da configuração da entidade `Veiculo`, que reúne chave primária, restrição de unicidade, tipos de coluna, restrições de verificação e as duas chaves estrangeiras:

```
modelBuilder.Entity<Veiculo>(entity =>
{
    entity.ToTable("Veiculo", t =>
    {
        t.HasCheckConstraint("CK_Veiculo_AnoFabricacao", "[AnoFabricacao] >= 1950");
        t.HasCheckConstraint("CK_Veiculo_Quilometragem", "[Quilometragem] >= 0");
    });

    entity.HasKey(v => v.VeiculoId);
    entity.Property(v => v.Placa).IsRequired().HasMaxLength(8);
    entity.Property(v => v.ValorDiaria).HasColumnType("decimal(10,2)");
    entity.HasIndex(v => v.Placa).IsUnique().HasDatabaseName("UQ_Veiculo_Placa");

    entity.HasOne(v => v.Fabricante)
        .WithMany(f => f.Veiculos)
        .HasForeignKey(v => v.FabricanteId)
        .HasConstraintName("FK_Veiculo_Fabricante")
        .OnDelete(DeleteBehavior.Restrict);
});
```

7.3. Registro do contexto na aplicação

Em `Program.cs`, o contexto é registrado no contêiner de injeção de dependência com a connection string definida em `appsettings.json`:

```
var connectionString = builder.Configuration.GetConnectionString("LocadoraConnection");

builder.Services.AddDbContext<ApplicationContext>(options =>
    options.UseSqlServer(connectionString));
```

7.4. Carga inicial de dados

Os dados fixos de apoio são criados junto com o banco, pela migração gerada a partir do método `HasData`:

```
modelBuilder.Entity<Categoria>().HasData(  
    new Categoria { CategoriaId = 1, Nome = "Economico", PercentualAjuste = 0m },  
    new Categoria { CategoriaId = 2, Nome = "Intermediario", PercentualAjuste = 15m },  
    new Categoria { CategoriaId = 3, Nome = "SUV", PercentualAjuste = 35m },  
    new Categoria { CategoriaId = 4, Nome = "Executivo", PercentualAjuste = 60m }  
);
```

8. Atendimento aos requisitos do enunciado

Requisito	Onde foi atendido
Todo veículo pertence a um fabricante	FK_Veiculo_Fabricante, coluna obrigatória Veiculo.FabricanteId
Veículo registra modelo, ano e quilometragem	Colunas Modelo, AnoFabricacao e Quilometragem, todas NOT NULL
Cliente com nome, CPF e e-mail	Colunas Nome, Cpf e Email (NOT NULL); CPF e e-mail com índice único
Aluguel atrelado a cliente e veículo em um período	FKs ClienteId e VeiculoId + DataRetirada e DataPrevistaDevolucao
Registro da devolução do veículo	Coluna DataDevolucao (nula em contrato aberto) e coluna Status
Quilometragem inicial e final do aluguel	Colunas QuilometragemInicial e QuilometragemFinal
Valor da diária e valor total da locação	Colunas ValorDiaria e ValorTotal, decimal(10,2)
Tradução do modelo pelo Entity Framework	Migração gerada a partir do ApplicationContext
Chaves primárias, estrangeiras e restrições	Seções 4, 5 e 6 deste documento
Classes de entidade em C#	Pasta Models: 6 classes + enumeradores de status
Mínimo de 5 entidades	6 entidades: Fabricante, Categoria, Veiculo, Cliente, Funcionario e Aluguel

9. Execução do projeto

Sequência utilizada para criar o banco a partir do modelo:

```
# 1) sobe o SQL Server
docker compose up -d

# 2) restaura os pacotes do projeto
dotnet restore

# 3) gera a migracao a partir do ApplicationContext e cria o banco
dotnet ef migrations add CriacaoInicial
dotnet ef database update

# 4) executa a aplicacao
dotnet run --urls http://0.0.0.0:5000
```

A aplicação expõe duas rotas de verificação desta etapa: GET /api/status, que confirma a conexão com o banco configurado, e GET /api/modelo, que lista as tabelas mapeadas com suas chaves primárias e estrangeiras, conforme montadas pelo ApplicationContext.

Caso o SQL Server Express esteja instalado localmente, basta substituir o valor de LocadoraConnection pela connection string LocadoraConnectionSqlExpress, já presente no appsettings.json.